

EXECUTIVE SUMMARY

Target: http://isolated-2.com
Scan ID: TORTURE-P3-2-ecd1aeef
Scan Date: 2026-03-04 17:31:04
Scan Duration: N/A
Total Requests Sent: 21
Avg Latency: N/A
Peak Concurrency: N/A
Findings: 2 vulnerabilities detected

Severity Breakdown: Critical: 2 | High: 0 | Medium: 0 | Low: 0
AI Inference Calls: 0
LLM Avg Latency: N/A
Circuit Breaker Activations: 0

- Overall Risk Level High due to two critical vulnerabilities identified in system
- Most Critical Category is Authentication/Authorization flaws allowing unauthorized access
- Immediate Actions required include patching authentication modules and implementing MFA
- Long-Term Recommendations involve regular security audits and continuous monitoring for new threats

EXECUTIVE STRATEGIC IMPACT

```
{
  "Strategic_Attack_Path_Analysis": [
    "An adversary could exploit the COMMAND_INJECTION vulnerability by injecting malicious commands into externally-influenced input, allowing them to execute arbitrary code on the vulnerable system.",
    "Following this initial foothold, they would leverage the SQL_INJECTION flaw to manipulate database queries, potentially gaining unauthorized access to sensitive data or escalating privileges within the compromised environment.",
    "By chaining these vulnerabilities together, an attacker could establish a persistent presence in the target network, escalate their control over critical systems, and ultimately achieve a high-impact objective such as full system compromise."
  ]
}
```

DETAILED FINDINGS

FILTER: INJECTION & FUZZING

Finding #1: OS Command Injection

CRITICAL

CWE: CWE-78
CVSS Score: 10.0 (CRITICAL)

THREAT SCORE: 100/100

Description:

- The target URL `http://test-target-torture-.com/api/v1/endpoint_0` is vulnerable to command injection attacks.
- Attackers can inject malicious commands via user input, potentially leading to unauthorized system access or data exfiltration.
- This vulnerability exists due to insufficient validation and sanitization of user-supplied data before execution.

Impact:

- Unauthorized commands could be executed with elevated privileges, compromising the integrity and confidentiality of sensitive data.
- An attacker may gain control over critical system functions, leading to potential denial-of-service or service disruption scenarios.
- The vulnerability poses a significant risk for both internal systems and external-facing services that interact with this endpoint.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'COMMAND_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

URL: http://test-target-torture-.com/api/v1/endpoint_0

Analysis: The application failed to properly sanitize user input, allowing malicious commands to be injected into API requests.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_0	test_payload_0	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: OS Command Injection

Raw Payload: test_payload_0

Encoded: 746573745f7061796c6f61645f30

Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_0'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_0 HTTP/1.1

Host: target

{}

Response:

HTTP/1.1 200 OK

Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- - Implement strict input validation and sanitization mechanisms to prevent command injection attacks by properly encoding user-supplied data before usage.
- - Use parameterized queries or prepared statements when executing SQL commands, eliminating the risk of injecting malicious code through string concatenation.
- - Apply least privilege principles for system processes interacting with this endpoint; restrict access rights to only necessary functions and resources.

Recommended Code Fix:

```
// Example secure code snippet using parameterized query (Java example)
```

```
PreparedStatement pstmt = conn.prepareStatement("SELECT * FROM users WHERE usern  
pstmt.setString(1, userInput.getUsername());  
pstmt.setString(2, userInput.getPassword());  
pstmt.executeUpdate();
```

Finding #2: SQL Injection

CRITICAL

CWE: CWE-89
CVSS Score: 10.0 (CRITICAL)

THREAT SCORE: 100/100

Description:

- Vulnerability 'SQL Injection' detected in target endpoint.

Impact:

- Unauthorized access to system resources confirmed.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SQL_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_1
URL: http://test-target-torture-.com/api/v1/endpoint_1
Analysis: The application failed to properly sanitize user input before using it in SQL queries.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_1	test_payload_1	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.
Result	200 OK	Server returns data for unauthorized request.

Payload Specifications:

Vector Category: SQL Injection
Raw Payload: test_payload_1
Encoded: 746573745f7061796c6f61645f31
Encoding Type: None

Reproduction Command:

```
curl -X POST 'http://test-target-torture-.com/api/v1/endpoint_1'
```

HTTP Traffic Snapshot:

Request:

```
POST http://test-target-torture-.com/api/v1/endpoint_1 HTTP/1.1
Host: target
{}
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}
```

Remediation:

- Implement strict input validation.

Recommended Code Fix:

```
```python
from flask import request, g, jsonify
import re

def sanitize_input(input_data):
 return re.sub(r'^\w\s', '', input_data)

@app.before_request
def before_request():
 user_input = request.args.get('user_input')
 sanitized_input = sanitize_input(user_input)
 g.user_input = sanitized_input

@app.route('/api/endpoint', methods=['GET'])
def api_endpoint():
 if 'user_input' in g:
 return jsonify({'result': f"Processed input: {g.user_input}"})
...```
```

# SCAN TIMELINE

[Orchestrator] TARGET\_ACQUIRED - 2026-03-04 17:31:04

[Sigma] JOB\_ASSIGNED - 2026-03-04 17:31:04

[Alpha] LOG - 2026-03-04 17:31:04

[Alpha] LOG - 2026-03-04 17:31:04

[Kappa] LOG - 2026-03-04 17:31:04

[Beta] LOG - 2026-03-04 17:31:05

[Kappa] LOG - 2026-03-04 17:31:05

[Beta] LOG - 2026-03-04 17:31:05

[Gamma] LOG - 2026-03-04 17:31:05

[Kappa] LOG - 2026-03-04 17:31:05

[Alpha] LOG - 2026-03-04 17:31:05

[Kappa] LOG - 2026-03-04 17:31:05

[Beta] LOG - 2026-03-04 17:31:05

[Beta] LOG - 2026-03-04 17:31:05

[Kappa] LOG - 2026-03-04 17:31:05

[Kappa] LOG - 2026-03-04 17:31:06

[Kappa] LOG - 2026-03-04 17:31:06

[Gamma] VULN\_CONFIRMED - 2026-03-04 17:31:06

[Gamma] VULN\_CONFIRMED - 2026-03-04 17:31:06

[Gamma] VULN\_CONFIRMED - 2026-03-04 17:31:06

[Kappa] GI5\_LOG - 2026-03-04 17:31:14